# Actividad Sesión No. 2

***estudiantes:***
*Angelie Rihanna Largo Trujillo*
*Nicoll Mariana Cifuentes García*

1. ***Pregunta orientadora:***

Imagine dos programas que resuelven exactamente el mismo problema. El primero necesita aproximadamente 1.000 operaciones y el segundo 1.000.000 de operaciones para procesar la misma cantidad de información.

¿Cuál elegiría para una aplicación que deberá procesar millones de registros?

2. **Reconociendo complejidades**

**Ejercicio 1**. Complejidad constante

public static void mostrarPrimero(int[] datos) { System.out.println(datos[0]); }

Si el arreglo tiene 10, 1.000 o 1.000.000 de elementos, el algoritmo consulta una sola posición. Complejidad: O(1).

**Ejercicio 2.** Complejidad lineal
public static void mostrarElementos(int[] datos) { for (int i = 0; i < datos.length; i++) { System.out.println(datos[i]); } }

El ciclo recorre todos los elementos una vez. Si n = 1.000, realiza aproximadamente 1.000 iteraciones. Complejidad: O(n).

**Ejercicio 3**.
Complejidad cuadrática public static void compararTodos(int[] datos) { for (int i = 0; i < datos.length; i++) { for (int j = 0; j < datos.length; j++) { System.out.println(datos[i] + " - " + datos[j]); } } }

Dos ciclos anidados producen aproximadamente n × n operaciones. Para n = 1.000, el orden de magnitud es 1.000.000. Complejidad: $O(n^2)$.

**Ejercicio 4.** Complejidad logarítmica

public static void reducir(int n) { while (n > 1) { n = n / 2; System.out.println(n); } }

En cada iteración el problema se reduce a la mitad. Complejidad: O(log n).

**Ejercicio 5.** Complejidad O(n log n) public static void ejemploNLogN(int n) { for (int i = 0; i < n; i++) { int valor = n; while (valor > 1) { valor = valor / 2; } } } El ciclo externo se ejecuta n veces y el interno aproximadamente log n veces. Complejidad: O(n log n).

3. **Clasificación de fragmentos**

| **Fragmento** | **Operaciones Aprox** | **Big O** | **Justificación** |
|---|---|---|---|
| Acceder a vector[5] | 1 | O(1) | Es un O(1), ya que como sabemos donde se encuentra el elemento lo buscamos directamente. |
| Recorrer un arreglo | n (Depende la longitud del arreglo) | O(n) | Es un O(n), ya que se busca |
| Dos ciclos anidados | $n^2$ | $O(n^2)$ | Al ser dos ciclos anidados, es n × n, lo que equivale a $n^2$.<br><br>También el hecho de que primero se realiza un ciclo de conteo y otro de verificación. |
| Dividir n entre 2 repetidamente | $\log_2(n)$ | O(log n) | En cada paso se divide a la mitad. |
| Ciclo n + reducción logarítmica | $n \times \log_2(n)$ | O(n log n) | El ciclo externo se ejecuta n veces mientras que el ciclo interno (Reducción logarítmica) se encarga de ir reduciendo a la mitad hasta llegar a 1 |

4. **Experimento: ¿cuál algoritmo escala mejor? Se compararán dos algoritmos: uno lineal O(n) y otro cuadrático $O(n^2)$.**

Algoritmo A – O(n)

```
public static long algoritmoLineal(int n) {
long contador = 0;
for (int i = 0; i < n; i++)
{ contador++; } return contador;
}
```

Algoritmo B – $O(n^2)$

```
 public static long algoritmoCuadratico(int n) {
 long contador = 0;
for (int i = 0; i < n; i++)
{ for (int j = 0; j < n; j++) { contador++; } }
return contador;
```

}

**Respuesta:** Escala mejor el O(n) ya que este siempre será un único paso; al saber dónde está ubicado el dato que necesitamos, no se requiere hacer una búsqueda exhaustiva, siendo más escalable en un contexto donde siempre se conozca en una lista dónde se encuentra el valor que necesitamos.

5. **Programa completo para medir tiempos**

Cree el archivo ComparacionAlgoritmos.java y escriba el siguiente programa

```
public class comparacionAlgoritmos {
public static void main(String[] args) {


int[] tamanos = {100, 1_000, 5_000, 10_000};


System.out.println("n\tO(n) ns\tO(n²) ns");

for (int n : tamanos) {
long inicioLineal = System.nanoTime();
algoritmoLineal(n);
long finLineal = System.nanoTime();
long tiempoLineal = finLineal - inicioLineal;
long inicioCuadratico = System.nanoTime();
algoritmoCuadratico(n);
long finCuadratico = System.nanoTime();
long tiempoCuadratico = finCuadratico - inicioCuadratico;
System.out.println(
n + "\t" + tiempoLineal + "\t" + tiempoCuadratico
);
}
}
public static long algoritmoLineal(int n) {
long contador = 0;
for (int i = 0; i < n; i++) {
contador++;
}
return contador;
}
public static long algoritmoCuadratico(int n) {
long contador = 0;
for (int i = 0; i < n; i++) {
for (int j = 0; j < n; j++) {
```

```
contador++;
}
}
return contador;
    }
    }
```

6. **Compilación y ejecución.**




7. **Registro de resultados**

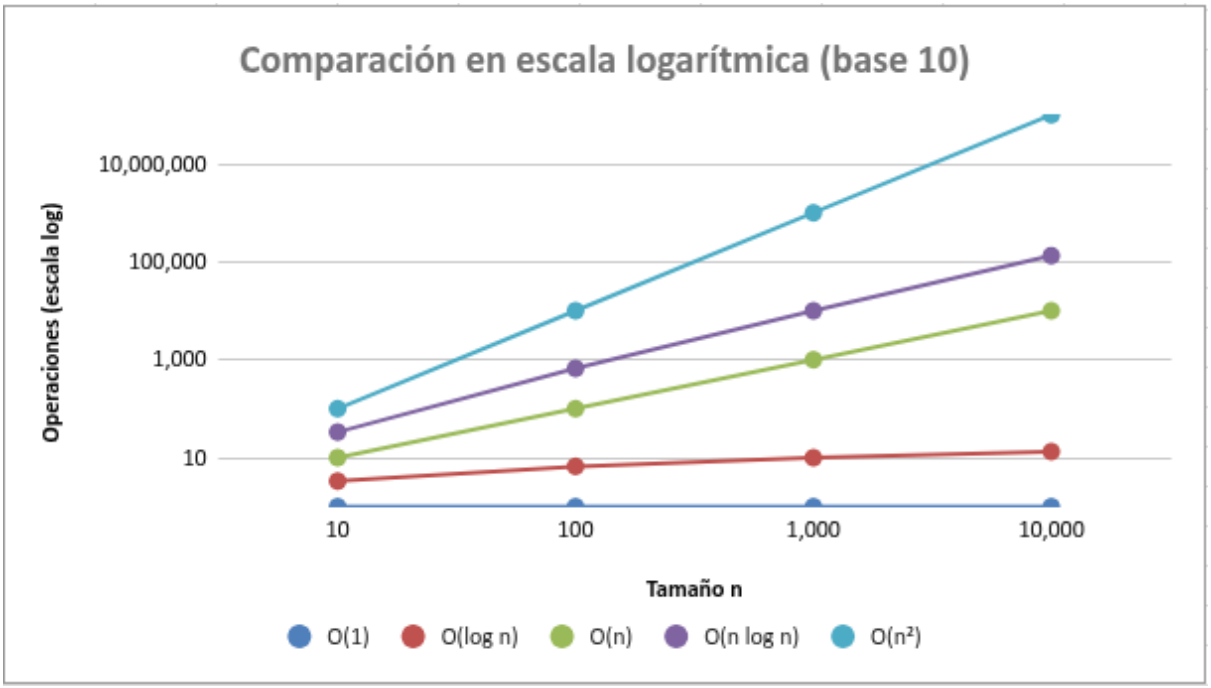
| Tamaño n | Tiempo O(n) | Tiempo O($n^2$) | ¿Cuál fue más rápido? |
|---|---|---|---|
| 100 | 3,720 | 139,138 | El O(n) |
| 1,000 | 14,829 | 3,275,864 | El O(n) |
| 5,000 | 67,117 | 12,532,426 | El O(n) |
| 10,000 | 150,817 | 7,368 | Anomalía del compilador de Java |

8. **Conteo Teorico**

| Tamaño n | O(1) | O(log n) | O(n) | O(n log n) | O($n^2$) |
|---|---|---|---|---|---|
| 10 | 1 | 3.32 | 10 | 33.22 | 100 |
| 100 | 1 | 6.64 | 100 | 664.39 | 10,000 |
| 1,000 | 1 | 9.97 | 1,000 | 9,965.78 | 1,000,000 |
| 10,000 | 1 | 13.29 | 10,000 | 132,877.12 | 100,000,000 |

9. **Representación Gráfica:**

Utilice Excel, Google Sheets o una herramienta equivalente. Cree las columnas n, O(n) y O(n²), registre los tamaños 10, 100, 1.000 y 10.000, y represente gráficamente el crecimiento. Observe especialmente qué ocurre con O(n²).




**10. Preguntas de análisis**

1. ¿De qué manera difieren el tiempo empleado en la ejecución y la cota superior Big O?

El tiempo de ejecución depende del computador, del lenguaje, de la JVM y de otros programas abiertos. Big O es un ejemplo corto y sencillo que puede presentar fallas en los cálculos o procesos. El tiempo solo dice cuánto se demora la ejecución en tu dispositivo, y Big O te dice cómo escala el algoritmo.

2. ¿Qué razones hacen que O(1) resulte tan eficiente?
Porque es constante; es como si buscáramos en una lista un elemento que ya sabemos dónde está: solo lo llamamos una vez, siendo una ejecución única, no como tratar de saber dónde está x dato de millones.

3. Al duplicar la cantidad de datos ingresados, ¿cómo reacciona un método O(n)?

El tiempo se duplica: crece en proporción directa. Con n datos hace n operaciones, y con 2n datos hace 2n.

4. Ante el incremento al doble del tamaño n, ¿qué comportamiento experimenta un algoritmo de tipo O(n²)?
El trabajo se cuadruplica, porque $(2n)^2 = 4n^2$

5. ¿A qué se debe que la iteración de bucles internos se vincule con O(n²)?

Porque por cada vuelta del ciclo externo (n veces) el interno se ejecuta completo (n veces). Los costos se multiplican:

6. ¿Qué patrón distintivo ayuda a identificar la presencia de O(log n)?
El problema se reduce a la mitad en cada paso (o se divide por una constante), como en while (n > 1) n = n / 2.

7. ¿Cuál procedimiento de la prueba demostró una tasa de incremento superior?
El cuadrático, O(n²). Al pasar de n = 1.000 a n = 10.000, el lineal debería multiplicarse por ~10 y el cuadrático por ~100. En conteo teórico, O(n²) llega a 100.000.000 de operaciones con n = 10.000, frente a 10.000 del lineal.

8. ¿Surgieron fluctuaciones en los tiempos entre las repeticiones efectuadas? ¿A qué motivo responde dicho fenómeno?
Sí. Por ejemplo, en la medición O(n²) con n = 10.000 salió menor (7.368 ns) que con n = 5.000. La causa de dicho fenómeno se debe principalmente al rendimiento del dispositivo y las capacidades del compilador de Java.

9. De requerir la manipulación de 10.000.000 de datos, ¿por cuál orden de magnitud optaría?
Elegiría el más bajo que el problema permita: O(log n) si basta con buscar (por ejemplo, búsqueda binaria en datos ordenados), O(n) si sabemos donde esta nuestro datoo.

10. ¿Qué importancia posee la evaluación de desempeño previo a elegir la solución algorítmica definitiva?

Evaluar el desempeño antes de elegir la solución final es importante porque con pocos datos casi cualquier algoritmo funciona, y los problemas de escalabilidad solo aparecen cuando n crece.

**11. Reto adicional**

for (int i = 0; i < n; i++) { System.out.println(i); } for (int j = 0; j < n; j++) { Ingeniería de Sistemas – Ingeniería de Software · III semester Mg. Sandra Esperanza Sánchez Barragán System.out.println(j); }

**Pregunta: ¿Es O(n²)? Justifique su respuesta. Pista: los ciclos son consecutivos, no están anidados.**

**Respuesta**: No es O(n²), los dos ciclos están uno después del otro (consecutivos), no uno dentro del otro. Primero termina el ciclo de `i` y solo después empieza el de `j`, así que sus costos se suman, no se multiplican: